```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

HttpClient client = new HttpClient                                  ❶
{                                                                   ❶
    BaseAddress = new Uri("https://example.com/rates/"),            ❶
};                                                                  ❶

app.MapGet("/", async () =>
{
    var response = await client.GetAsync("latest");                 ❷

    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync();
});
app.Run();
```

❶ A single instance of HttpClient is created for the lifetime of the app.

❷ Multiple requests use the same instance of HttpClient.

This solves the problem of socket exhaustion. As you're not disposing of the `HttpClient`, the socket is not disposed of, so you can reuse the same port for multiple requests. No matter how many times you call the API in the preceding example, you will use only a single socket. Problem solved!

Unfortunately, this introduces a different problem, primarily related to DNS. DNS is how the friendly hostnames we use, such as manning.com, are converted to the Internet Protocol (IP) addresses that computers need. When a new connection is required, the `HttpClient` first checks the DNS record for a host to find the IP address and then makes the connection. For subsequent requests, the connection is already established, so it doesn't make another DNS call.

For singleton `HttpClient` instances, this can be a problem because the `HttpClient` won't detect DNS changes. DNS is often used in cloud environments for load balancing to do graceful rollouts of deployments.[1] If the DNS record of a service you're calling changes during the lifetime of